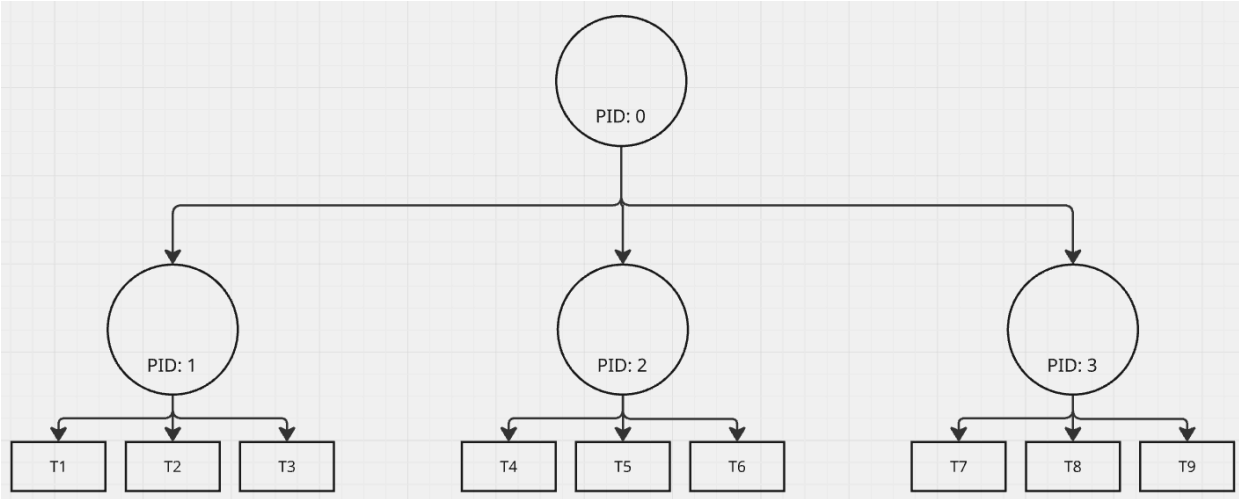
1.

a.



Parent:
Global_var = 3
Static_var = 3
Local_var = 1
Dynamic_var = 1

Child:
Global_var = 0
Static_var = 0
Local_var = 1
Dynamic_var = 1

b.

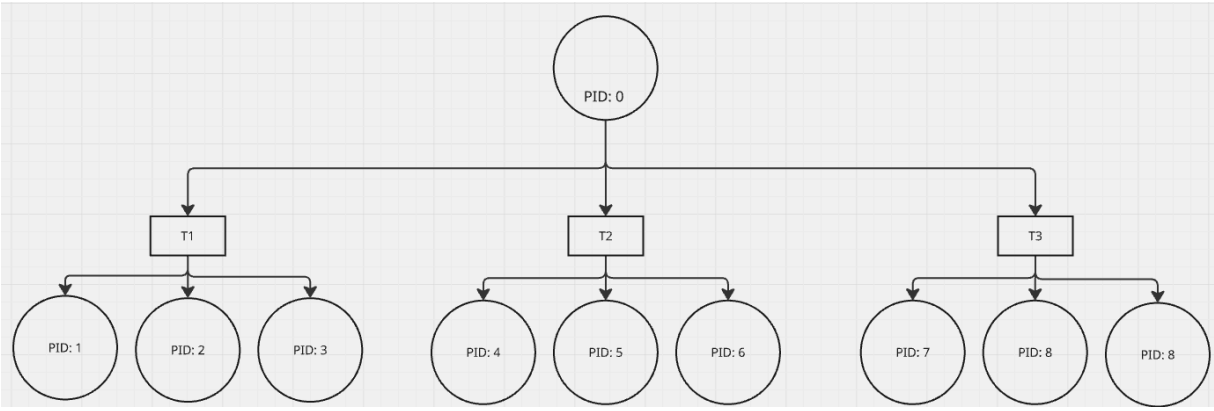
c.

```
ewebb11@csx4:~$ ./question1
PID=3642107 TID=123712604993216: global_var=1 static_var=1 local_var=1 *dynamic_var=1
PID=3642107 TID=123712596600512: global_var=2 static_var=2 local_var=1 *dynamic_var=1
PID=3642107 TID=123712451901120: global_var=3 static_var=3 local_var=1 *dynamic_var=1
PID=3642109 TID=123712604993216: global_var=1 static_var=1 local_var=1 *dynamic_var=1
PID=3642108 TID=123712596600512: global_var=1 static_var=1 local_var=1 *dynamic_var=1
PID=3642108 TID=123712588207808: global_var=3 static_var=3 local_var=1 *dynamic_var=1
PID=3642108 TID=123712604993216: global_var=2 static_var=2 local_var=1 *dynamic_var=1
PID=3642109 TID=123712596600512: global_var=2 static_var=2 local_var=1 *dynamic_var=1
PID=3642109 TID=123712588207808: global_var=3 static_var=3 local_var=1 *dynamic_var=1
Child PID=3642107 final values: global_var=3 static_var=3
Child PID=3642108 final values: global_var=3 static_var=3
Child PID=3642109 final values: global_var=3 static_var=3
Parent PID=3642106 final values: global_var=0, static_var=0
```

The output displays the variable values for each thread, then the final values for each process.

2. .

a.



Parent:
Global_var = 3
Static_var = 3
Local_var = 1
Dynamic_var = 1

Child:
Global_var = 4
Static_var = 4
Local_var = 2
Dynamic_var = 2

b.

```
ewebb11@csx4:~$ ./question2b
Child PID=3644601 final values: global=2 static=2 local=2 dynamic=2
Child PID=3644602 final values: global=2 static=2 local=2 dynamic=2
Child PID=3644603 final values: global=2 static=2 local=2 dynamic=2
PID=3644597, TID=125103171958464 global=2 static=2 local=1 dynamic=1
Child PID=3644604 final values: global=3 static=3 local=2 dynamic=2
Child PID=3644605 final values: global=3 static=3 local=2 dynamic=2
Child PID=3644606 final values: global=3 static=3 local=2 dynamic=2
Child PID=3644607 final values: global=4 static=4 local=2 dynamic=2
PID=3644597, TID=125103163565760 global=3 static=3 local=1 dynamic=1
Child PID=3644608 final values: global=4 static=4 local=2 dynamic=2
Child PID=3644609 final values: global=4 static=4 local=2 dynamic=2
PID=3644597, TID=125103155173056 global=3 static=3 local=1 dynamic=1
Parent PID=3644597 final values: global=3 static=3
```

The output displays the variable values for each thread and the final values for each process.

c. I would modify the given code by using static __thread int static_var = 0; to have each thread have its own instance of static_var.

```
ewebb11@csx4:~$ ./question2c
Child PID=3645615 final values: global=2 static=2 local=2 dynamic=2
Child PID=3645616 final values: global=2 static=2 local=2 dynamic=2
Child PID=3645617 final values: global=2 static=2 local=2 dynamic=2
PID=3645611 TID=125673534387904 global=2 static=1 local=1 dynamic=1
Child PID=3645618 final values: global=3 static=2 local=2 dynamic=2
Child PID=3645619 final values: global=3 static=2 local=2 dynamic=2
Child PID=3645620 final values: global=3 static=2 local=2 dynamic=2
Child PID=3645621 final values: global=4 static=2 local=2 dynamic=2
PID=3645611 TID=125673517602496 global=3 static=1 local=1 dynamic=1
Child PID=3645622 final values: global=4 static=2 local=2 dynamic=2
Child PID=3645623 final values: global=4 static=2 local=2 dynamic=2
PID=3645611 TID=125673525995200 global=3 static=1 local=1 dynamic=1
Parent PID=3645611 final value: global=3 static=0
```

3.

a. Possible Outputs:

1) If thread_function1 runs first
Main process started
Thread 1 created

2) If thread_function2 runs first or overlaps before execvp()
Main process started
Thread 1 created
Thread 2 created
Thread 2 is done

3) if fork() happens before execvp() executes
Main process started
Thread 1 created
Thread 2 created
Parent process continuing
Child process executing

4) If execvp() fails
Main process started
Thread 1 created
execvp: No such file or directory
Thread 1 is done
Thread 2 created
Thread 2 is done
Parent process continuing
Child process executing
Main process is done

b. Issue 1: pthread_t thread_id reused for two threads
   Solution: use two different pthread_t variables
   Issue 2: execvp() in a thread replaces the entire process
   Solution: create a separate process instead of using exec in the thread.
   Issue 3: no pthread_exit() or return in thread_function1
   Solution: use pthread_exit(NULL); if exec fails
   Issue 4: Mixing pthread_create() and fork()
   Solution:move fork() before starting threads

4.

```
ewebb11@csx4:~$ ./question4
Thread finished: global=69412, static=71036, local=100042, dynamic=100000
Thread finished: global=62372, static=63997, local=100042, dynamic=100000
Thread finished: global=109833, static=112032, local=100042, dynamic=100000
Thread finished: global=120307, static=122505, local=100042, dynamic=100000
Thread finished: global=120932, static=123131, local=100042, dynamic=100000
Main thread finished: global=120932 static=123131 local=42
```

The program creates five threads and each performs 100,000 iterations that increment the 4 variables.
local_var
Threads start with local_var = 42 then increments 100,000 times so 100000 +42 = 100042
Dynamic_var each thread
Threads dynamically allocates its own integer (malloc)
Starts at 0 and is incremented 100,000 times so = 100,000
global_var and static_var
Both are shared and visible to all threads
Each attempt increments 100000 times, but they are not synchronized, so some increments are lost so you don't get the expected value of 500000.

5. alue = 1
   alue = 2
   The variable value is global and starts at 0.
   Thread 1 is created
   Thread 2 is created
   Each one runs func1, which increments the value by 1, then prints the value.
   Giving the output
   alue = 1
   alue = 2
   If both threads read value 0 before the other increments, then you would have a race condition problem and get
   alue = 1
   alue = 1

6. Thread1 (before fun1_1 call): Count value is = 11
   Thread1 (inside fun1_1): Count value is = 12
   Thread1 (after fun1_1 call): Count value is = 12
   Thread2: Count value is = 13

   The variable count is global and starts at 10
   Thread 1 is created
   Thread 2 is created
   Thread 1 runs fun1() which increments count value by 1
   Then prints the value at 11
   Then calls fun1_1() which increments count value by 1
   Then prints the value at 12
   Then fun1() prints the value after fun1_1() at 12
   Then thread 2 runs fun2() which increments the value to 13
   Then prints the value at 13

   There are other possible outputs based on thread timings.

7. Total time on 1 processor
   = 40ns + 60ns = 100ns

   Total time on 4 processors
   Sequential = 40ns
   Parallelizable = 60/4 = 15ns
   = 40ns + 15ns = 55ns

   Speedup
   100/55 = 1.81818182

8. Total time on 1 processor
   100ns + 300ns = 400ns

   Speedup
   = 1 / ((.25) + (.75/4)) = 2.28571429